

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Positional Encoding in Transformer



Sachin Soni

Follow

11 min read · Sep 3, 2024



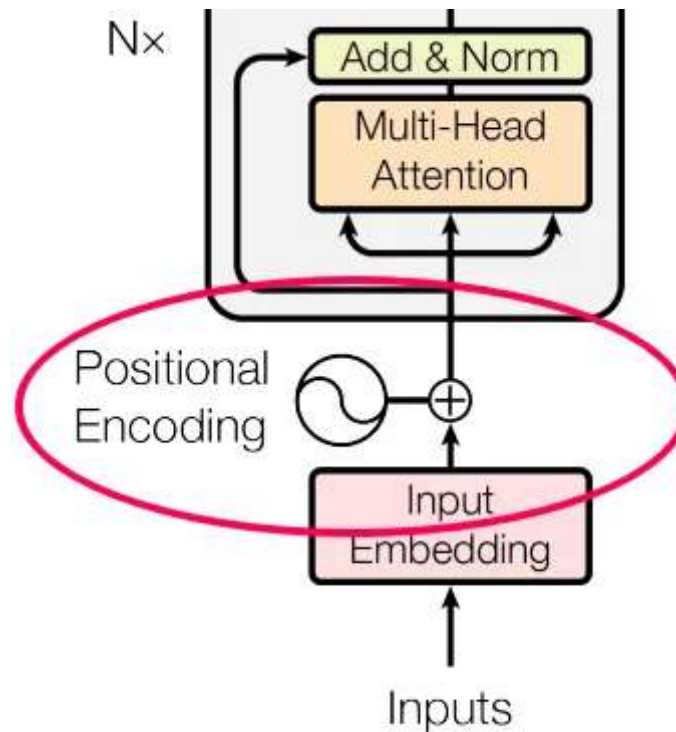
51



5



In our previous discussions on self-attention and multi-head attention, we saw how transformers excel at capturing relationships between words by focusing on different parts of a sequence. However, these mechanisms alone don't tell the model where each word is in a sentence. Since word order is crucial for understanding meaning, transformers need a way to know the position of each word. This is where positional encoding comes in. It adds information about the order of words, allowing the model to make sense of sequences like sentences. In this blog, we'll explore how positional encoding works and why it's key to making transformers so powerful.



[image source](#)

*Before we dive into the details of positional encoding, I want to extend my heartfelt gratitude to my mentor, **Nitish Sir**. His exceptional guidance and teachings on the [CampusX YouTube channel](#) have been instrumental in shaping my understanding of this complex topic. With his support, I've embarked on this journey of exploration and learning, and I'm excited to share my insights with you all. Thank you, Nitish Sir, for being an inspiration and mentor!*

The best part of the self-attention mechanism is its ability to generate dynamic contextual embeddings based on the context in which a particular word is used. Additionally, a major advantage of self-attention is that it allows all contextual embeddings to be calculated in parallel, making it possible to process large documents quickly. *However, this parallel processing comes with a significant drawback: the self-attention module cannot capture the order of words in a sentence.* For example, consider these two sentences:

1. Ravi killed the lion.
2. The lion killed Ravi.

If you pass both sentences through a self-attention block, it won't be able to distinguish between them, as it doesn't capture the order of the words.

This limitation can lead to misunderstandings in meaning, as the module treats sentences with the same words in different orders as identical. To address this issue, positional encodings are introduced in the transformer architecture. **Positional encodings provide information about the order of words in a sentence, ensuring that the model understands the sequence and maintains the correct context.**

Now, let's talk about the solution: **positional encoding**. Instead of jumping straight into the details of positional encoding, I'm going to take you through a "first principles" approach. This means we'll start with a very basic solution, identify its shortcomings, and then refine it step by step until we arrive at the concept of positional encoding. Trust me, this method will help you understand the topic much more clearly.

Let's begin with a simple sentence: "Ravi killed the lion" This sentence has four words. The problem is that our self-attention block doesn't inherently understand the order in which these words appear. Somehow, we need to convey to the self-attention block that "Ravi" comes first, "killed" comes second, "the" comes third, and so on.

A straightforward solution might be to assign a position number to each word. For example, "Ravi" is 1, "killed" is 2, and so forth. Then, we can embed this positional information into our word embeddings. Suppose our word embeddings have 512 dimensions. We could add a 513th dimension to store this positional number. This way, we're telling the self-attention block the position of each word in the sequence.

In the approach where we append simple positional numbers to our embeddings, there are several significant problems that make this solution impractical. Let's break down the first major issue:

Problem 1: Unbounded Growth of Numbers

One of the biggest challenges with this method is that the numbers we append can grow without bounds. For instance, if you have a short sentence with four words, the highest position number would be 4. But what if you were processing a whole book? A typical book might have 500 pages, with approximately 200 words per page. That means you could end up with position numbers reaching up to 100,000 or even higher.

Such large numbers pose a significant problem in the context of neural networks, particularly because they rely on the principle of backpropagation for training. **Backpropagation tends to work best with numbers in the range of -1 to 1.** When you start introducing very large numbers like 100,000 or more, it can lead to instability during training. **This instability might manifest as issues like vanishing gradients or exploding gradients, which can severely hamper the training process.**

One might think that normalizing these numbers could solve the problem. For example, if you have a sentence with four words, you could divide each position by the total number of words. In a sentence with four words, the last word would have a normalized position of 1 ($4/4$). The same approach could be applied to a book with 100,000 words, where the last word would also be normalized to 1 ($100,000/100,000$).

However, normalization introduces another problem: inconsistency.

Imagine you have two sentences:

- “Thank you” with positions normalized to $1/2$ and $2/2$.
- “Ravi killed the lion” with positions normalized to $1/4$, $2/4$, $3/4$, and $4/4$.

In the first sentence, the second word “you” has a normalized value of 1, while in the second sentence, the second word “killed” has a normalized value of 0.5. Even though both words are in the second position of their respective sentences, they end up with different positional values. This inconsistency can confuse the neural network, as it won’t have a clear understanding of what the second position actually represents.

Problem 2: Discrete Numbers

The second significant issue with the simple positional numbering approach is that the numbers we’re using, like 2, 3, 4, etc., are discrete. Neural networks typically perform better when they can work with continuous, smooth transitions rather than abrupt, discrete jumps. The use of discrete numbers can lead to numerical instability during training, negatively impacting the gradient flow.

In other words, when the network encounters such sharp transitions between positions, it struggles to maintain stable learning, which is crucial for effective training. This instability is another reason why the simple counting approach is not suitable.

Problem 3: Inability to Capture Relative Positions

The third problem is that this approach fails to capture relative positions. Let's break this down:

- **Absolute Positioning:** In your simple approach, each word gets a unique position number. For instance, in the sentence “Ravi killed the lion,” “Ravi” is assigned position 1, “killed” is assigned position 2, “the” is position 3, and “lion” is position 4. This is fine for capturing **absolute positions**.
- **Relative Positioning:** However, the model fails to understand the relative distances between these positions. For example, the word “the” follows “Ravi,” but how far apart are they? The model doesn't easily grasp this because it's working with discrete values. It's not straightforward for a neural network to compute and understand the distance between positions 1 and 3, especially when dealing with discrete numbers.

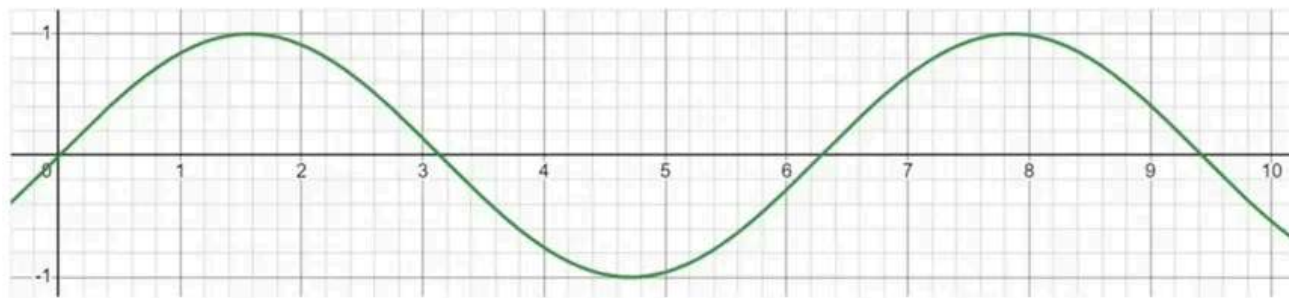
The key point here is that while our approach can capture absolute positions, it doesn't effectively convey how close or far apart different positions are from one another. This understanding of distance between positions — what we call **relative positioning** — is crucial for many tasks in natural language processing (NLP).

A Potential Solution :

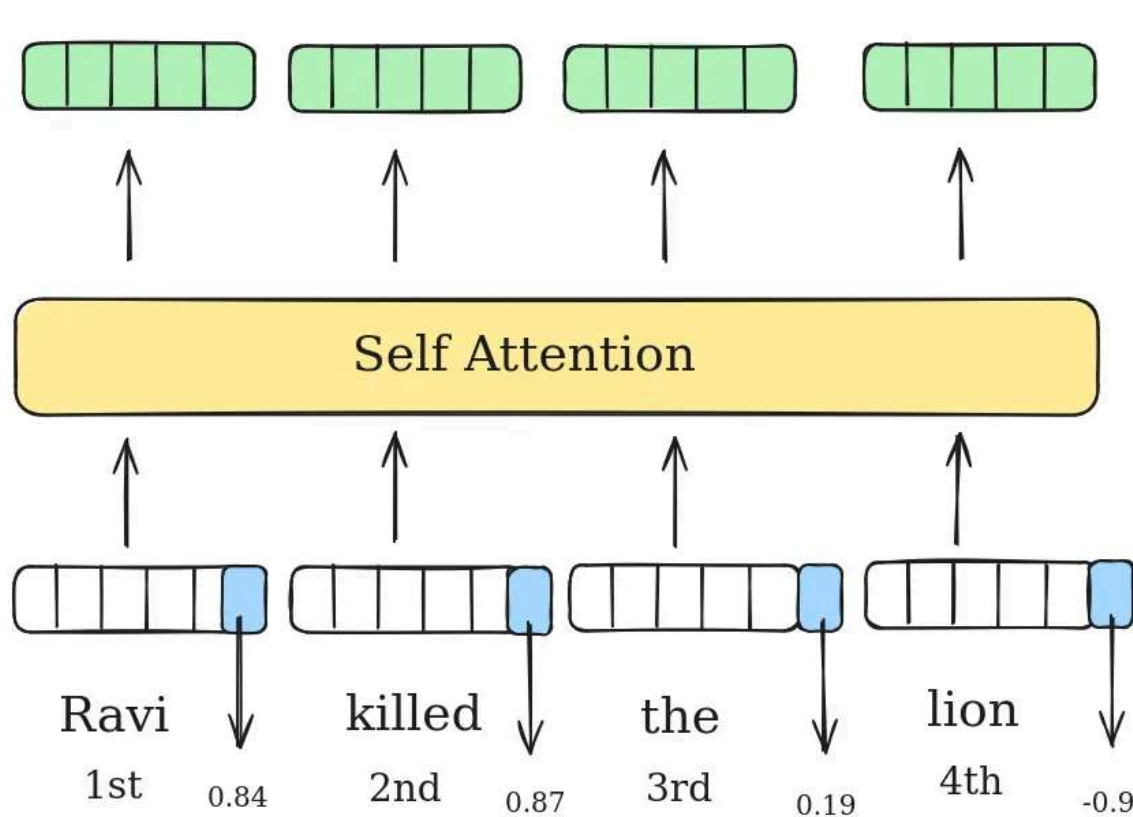
Now, to solve these issues, what we need is a function that:

1. **Is Bounded:** The values it produces should lie within a specific range.
2. **Is Continuous:** The function should provide smooth transitions between values.
3. **Is Periodic:** The function should repeat its pattern over intervals, helping capture relative positions effectively.

If you've studied mathematics, you might already be thinking of trigonometric functions like sine and cosine, which naturally exhibit these properties. These functions are bounded (values range between -1 and 1), continuous, and periodic.



Graph of Sine Function



Calculating position value using sine function:- ↓

$$y = \sin(\text{position})$$

$$y = \sin(1) = 0.84$$

$$y = \sin(2) = 0.91$$

$$y = \sin(3) = 0.14$$

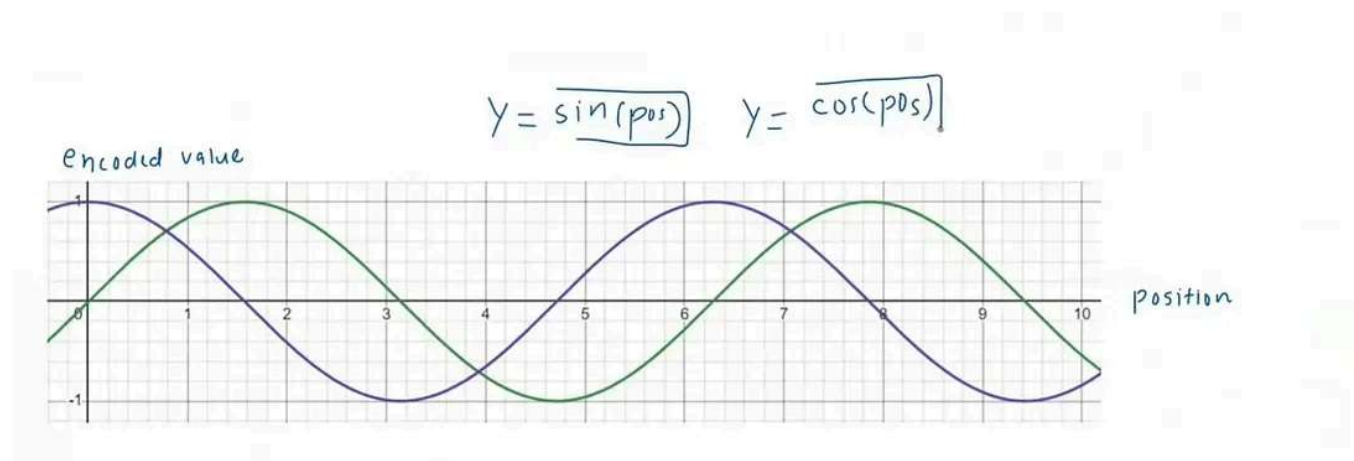
$$y = \sin(4) = -0.76$$

Calculating position value using sine function and appending the value in the last of embedding vector of each word.

While the sine function approach addresses several problems with simple positional numbering, it introduces a significant new issue: **periodicity**.

Since **sine and cosine functions repeat their values**, different positions in a sentence might end up with the same encoding. For example, both the 3rd and 35th positions might have identical sine values, causing the model to mistakenly treat these different positions as the same. This can lead to confusion and errors in understanding the sequence.

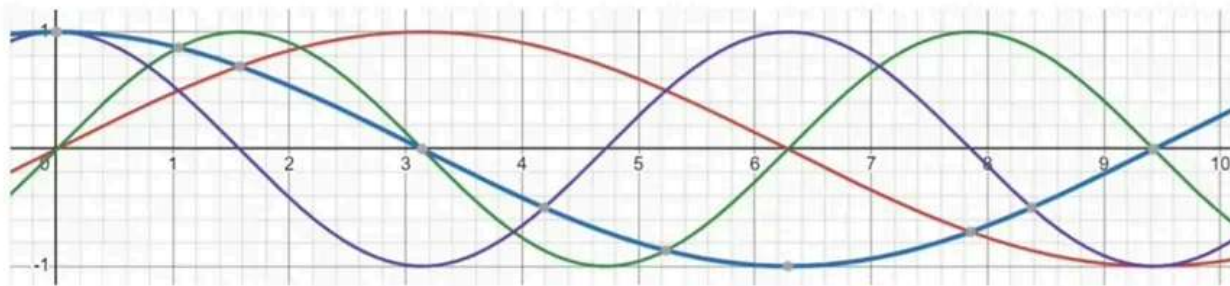
To solve the issue of periodicity in positional encoding, instead of using a single trigonometric function, we can use both sine and cosine functions. By doing so, each word's position is represented as a vector rather than a single scalar value.



Graph of both sine and cosine function. Here x axis represents position and y axis represents encoded value.

For instance, instead of using just $y = \sin(\text{position})$, you use both $y = \sin(\text{position})$ and $y = \cos(\text{position})$. This gives each position two values, creating a vector. For example, the 1st position might be represented by $(\sin(1), \cos(1))$, and the 2nd position by $(\sin(2), \cos(2))$.

To further reduce the chance of different positions having the same encoding, you can add more pairs of trigonometric functions with different frequencies. For example, you can include $\sin(\text{position}/2)$ and $\cos(\text{position}/2)$ as additional functions. This results in a higher-dimensional vector, such as $(\sin(1), \cos(1), \sin(1/2), \cos(1/2))$, making it even less likely for different positions to have the same encoding values.



Graph of multiple sine and cosine function

By increasing the number of pairs, you effectively reduce the probability that two distinct positions will have identical encodings, even in long sentences or large documents.

Positional Encoding in original research paper “Attention Is All You Need” :

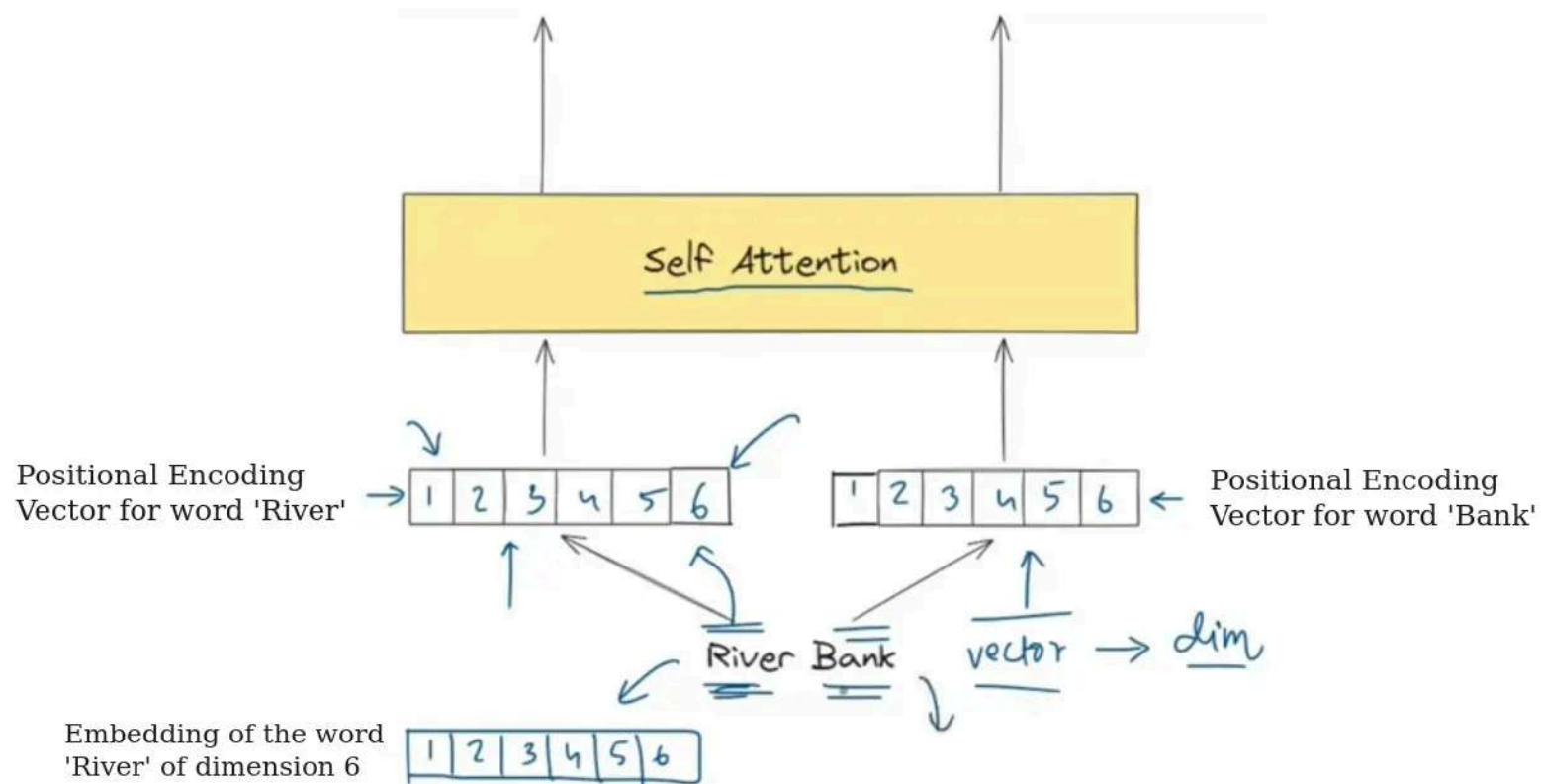
Since we have discussed the basics of positional encoding, now let's dive into the actual solution presented in the “Attention Is All You Need” paper. We'll explore how positional encoding is implemented to address the issue of capturing word order in sequences.

In the transformer architecture, positional encoding is essential because it allows the model to understand the position of each word in a sequence. Let's take an example sentence: “river bank.” For this sentence, you need to calculate positional encodings for each word — “river” and “bank” — before passing them through the self-attention block.



Positional Encoding Vector Dimensions

The key point is that the **positional encoding vector for each word will have the same dimension as its embedding vector**. If the embedding vector for “river” has 6 dimensions, its positional encoding vector will also have 6 dimensions. The same applies to “bank.” Each positional encoding vector is a set of numbers, and its dimension matches that of the word embeddings.



Combining Positional Encodings with Word Embeddings :

Once you have the positional encoding vectors, you add them to the word embedding vectors. This addition combines the positional information with the word embeddings, giving you a combined vector that contains both types of information. The result is still a 6-dimensional vector, which is the sum of the embedding vector and the positional encoding vector.

Why Addition Over Concatenation?

The authors of the “Attention Is All You Need” paper opted for addition instead of concatenation. Concatenating would double the dimensionality of the vectors (from 6 to 12 in this case), leading to an increase in the number of parameters and, consequently, the training time. Addition keeps the dimensionality consistent and avoids this increase, making the training process more efficient.

Now, let's address how to compute the values within the positional encoding vectors. Suppose we need to calculate the 6-dimensional positional encoding

vector for the word “river,” which is at position 1. We’ll use sine and cosine functions for this calculation.

Steps to Compute Positional Encoding Values:

1. Dimensional Calculation:

- For each dimension of the positional encoding vector, we’ll use a sine or cosine function.
- For a 6-dimensional vector, we’ll use three sine and three cosine functions, each with different frequencies.

2. Applying Sine and Cosine Functions:

- For the first two dimensions, take a sine and cosine function.
- For each dimension ‘i’ in the vector, compute values using the formula:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

where **pos** is the position of the word, '**i**' is the dimension index, and **d** is the total dimension of the positional encoding.

Example for Position 1:

- For each dimension, calculate the sine and cosine values using the above formulas.
- For position 1, you will calculate values using the specific frequencies corresponding to each dimension.

For Other Positions:

- Repeat the same process for other positions (e.g., position 2 for the word “bank”). Each position will have its unique set of values based on the formulas. See below diagram for example :

Calculating positional vector
of word River in 0th position
and since its embedding vector
dimension is 6, hence it is
represented by 6th dimensional
positional encoding



$$\begin{aligned}
 &\boxed{\text{for } i = 0} \rightarrow \text{pos} = 0 \\
 &PE(0, 0) = \sin\left(\frac{0}{10000^2}\right) = \boxed{0} \\
 &PE(0, 1) = \cos\left(\frac{0}{10000^2}\right) = \boxed{1}
 \end{aligned}$$

Calculating positional vector
of word Bank in 1st position
and since its embedding vector
dimension is 6, hence it is
represented by 6th dimensional
positional encoding



$$\begin{aligned}
 PE(1, 0) &= \sin\left(\frac{1}{10000^2}\right) = 0.84 \\
 PE(1, 1) &= \cos\left(\frac{1}{10000^2}\right) = 0.54
 \end{aligned}$$

for $i=1$ ✓

$$PE(0, 2) = \sin(0 / 10000^{1/3}) = 0$$

$$PE(0, 3) = \cos(0 / 10000^{1/3}) = 1$$

for $i=2$ ✓

$$PE(0, 4) = \sin(0 / 10000^{2/3}) = 0$$

$$PE(0, 5) = \cos(0 / 10000^{2/3}) = 1$$

for $i=1$

$$PE(1, 2) = \sin(1 / 10000^{1/3}) = 0.04$$

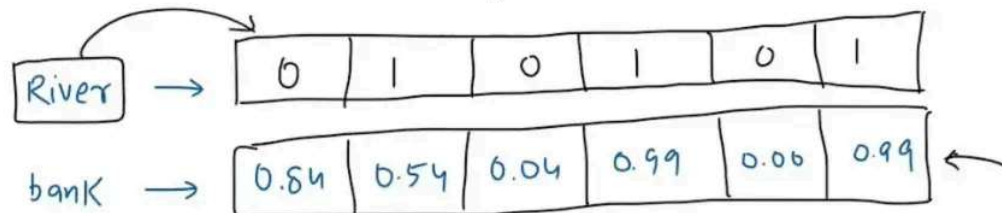
$$PE(1, 3) = \cos(1 / 10000^{1/3}) = 0.99$$

for $i=2$

$$PE(1, 4) = \sin(1 / 10000^{2/3}) = 0.00$$

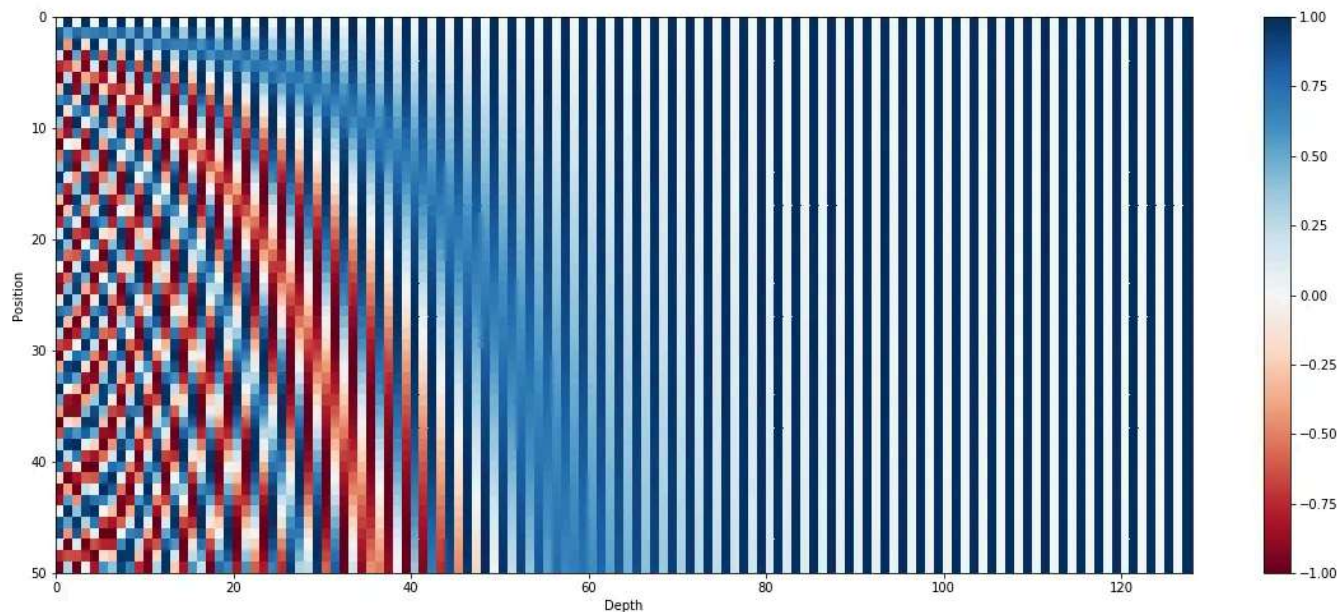
$$PE(1, 5) = \cos(1 / 10000^{2/3}) = 0.99$$

At last, we get positional encoding of word in 6th dimension.



Demonstrating the actual calculation of positional encoding by taking River Bank example

Heatmap of positional encodings for a sentence with 50 words :



[image source](#)

The graph visualizes positional encodings for a sentence with 50 words, each word represented by a 128-dimensional vector. The vectors are displayed using a heatmap where colors represent different values.

1. Observations from the Graph:

- **Frequency Patterns:** Each positional encoding vector is visualized across 128 dimensions. The color pattern (white and blue) shows the encoding values.

- **Dimensional Analysis:** Lower dimensions exhibit more variation with noticeable color changes, while higher dimensions show less variation.
- **Lower Dimensions:** The variation is due to the higher frequency sine and cosine functions used for lower dimensions.
- **Higher Dimensions:** Less variation is observed due to lower frequency functions used for higher dimensions.

2. Frequency and Dimension Relationship:

- **Higher Frequency for Lower Dimensions:** Lower dimensions use higher frequency sine and cosine waves, causing more noticeable changes in the encoding values.
- **Lower Frequency for Higher Dimensions:** Higher dimensions use lower frequency waves, resulting in less variation in encoding values.

3. Impact of Increasing Words:

- If the number of words increases (e.g., from 50 to 100), the higher dimensions will also show variation because the range of positions becomes larger.

- The graph would reflect more variation across all dimensions if more words are included in the sentence, leading to a fuller utilization of all dimensions.

Understanding Relative Position Encoding with Sine-Cosine Functions :

- The sine-cosine encoding creates vectors in such a way that applying a linear transformation (matrix) to a vector from a certain position can map it to another position relative to the original.
- For example, if applying a matrix to vector v_{10} maps it to v_{20} , the same matrix applied to v_{30} will map it to v_{40} , and so forth. This demonstrates how relative positions are maintained through linear transformations.
- Each linear transformation in the sine-cosine encoding system can cover a specific delta or distance between positions.
- The capability of the matrix to handle various deltas means that the encoding effectively captures relative positions between words.

For a detailed explanation of how the linear transformation works to calculate relative positions, check out [this blog](#).

References :